LAVORARE CON FILEDI TESTO NELLA CLI

Obiettivi:

- **Parte 1: Editor di Testo Grafici**
- **Parte 2: Editor di Testo da Riga di Comando**
- **Parte 3: Lavorare con i File di Configurazione**

PARTE 1: EDITOR DI TESTO GRAFICI

Sei riuscito a trovare subito space.txt?

no ma da file explorer quando faccio due click riesco ad aprilo con scite

SciTE sta guardando nella directory corretta (/home/analyst), space.txt non viene visualizzato. Questo perché SciTE cerca estensioni note e .txt non è una di queste. Per visualizzare tutti i file, fai clic sul menu a discesa nella parte inferiore della finestra Open File e seleziona All Files (*).

Perché il prompt non viene mostrato nel terminale?

Quando digiti scite space.txt, il terminale avvia l'applicazione SciTE come un **processo in primo piano** (foreground process). Finché SciTE è aperto, esso "occupa" la shell del terminale per comunicare con il sistema operativo, impedendo la comparsa del prompt per nuovi comand

PARTE 2: EDITOR DI TESTO DA RIGA DI COMANDO

Quale carattere usa nano per rappresentare che una linea continua oltre i bordi dello schermo?

Il carattere che **nano** utilizza per rappresentare che una riga di testo continua oltre i bordi dello schermo è il simbolo **>**

PARTE 3: LAVORARE CON I FILE DI CONFIGURAZIONE

Perché i file di configurazione delle applicazioni utente sono salvati nella directory home dell'utente e non sotto /etc con tutti gli altri file di configurazione a livello di sistema?

I file di configurazione delle applicazioni utente sono salvati nella home anziché in /etc per tre ragioni principali:

1. **Personalizzazione e Isolamento**: Ogni utente può avere le proprie impostazioni (come il colore del terminale o le scorciatoie da tastiera) senza influenzare o sovrascrivere quelle degli altri utenti del sistema.
2. **Permessi di Sicurezza**: La directory /etc è di proprietà dell'utente root. Se le configurazioni utente fossero lì, un utente normale non potrebbe modificarle senza privilegi amministrativi (sudo). Salvandole nella home, l'utente ha il pieno controllo sui propri file.
3. **Portabilità**: Se un utente sposta la propria cartella home su un altro computer Linux, porterà con sé tutte le sue configurazioni personali senza dover toccare i file di sistema del nuovo computer.

Anche la finestra del terminale che era già aperta ha cambiato colore da verde a rosso? Spiega.

No, il terminale che era già aperto prima della modifica non ha cambiato colore. Questo perché il file di configurazione .bashrc viene caricato dalla shell **solo quando viene avviata una nuova sessione di terminale**.
Le modifiche apportate al prompt vengono quindi applicate **solo alle nuove shell aperte dopo il salvataggio del file**, mentre le sessioni già attive continuano a utilizzare la configurazione precedente finché non vengono chiuse o ricaricate manualmente.

BONUS

Qual è la funzione del comando `cp`?

Il comando cp serve per copiare file e directory

Nel manuale la spiegazione principale è nella sezione **DESCRIPTION**: **Copy SOURCE to DEST, or multiple SOURCE(s) to DIRECTORY.**

copia una **sorgente (SOURCE)** in una **destinazione (DEST)**, oppure copia **più sorgenti** dentro una **directory**.

```
NAME
       cp - copy files and directories

SYNOPSIS
       cp [OPTION]... [-T] SOURCE DEST
       cp [OPTION]... SOURCE... DIRECTORY
       cp [OPTION]... -t DIRECTORY SOURCE...

DESCRIPTION
       Copy SOURCE to DEST, or multiple SOURCE(s) to DIRECTORY.

       Mandatory arguments to long options are mandatory for short options too.

       -a, --archive
              same as -dR --preserve=all

       --attributes-only
              don't copy the file data, just the attributes

       --backup[=CONTROL]
              make a backup of each existing destination file

       -b     like --backup but does not accept an argument

       --copy-contents
              copy contents of special files when recursive
```

Quale comando useresti per trovare maggiori informazioni sul comando pwd? Qual è la funzione del comando pwd?

```
NAME
       pwd - print name of current/working directory

SYNOPSIS
       pwd [OPTION]...

DESCRIPTION
       Print the full filename of the current working directory.

       -L, --logical
              use PWD from environment, even if it contains symlinks

       -P, --physical
              resolve all symlinks
```

Per trovare informazioni sul comando si usa **man pwd**, che apre la pagina di manuale del programma.

Dal manuale emerge subito la funzione. La sezione **NAME** dice:
pwd - print name of current/working directory

E nella **DESCRIPTION** viene chiarito meglio:
Print the full filename of the current working directory.

Quindi pwd serve semplicemente a **mostrare il percorso completo della directory in cui ti trovi in quel momento** nel terminale.

Elenca alcune sezioni incluse in una pagina man.,

Le più importanti per capire rapidamente un comando sono quasi sempre **NAME, SYNOPSIS, DESCRIPTION e OPTIONS**, perché permettono di capire subito *cosa fa* e *come si usa*.

NAME indica il nome del comando e una breve descrizione della sua funzione.
SYNOPSIS mostra **come si usa il comando**, cioè la sintassi: ordine di argomenti, opzioni e parametri.
DESCRIPTION spiega **cosa fa il comando** e come funziona in modo più dettagliato.
OPTIONS descrive le **opzioni disponibili** (flag come -a, -f, --help) e il loro effetto.

NAME, SYNOPSIS, DESCRIPTION, EXAMPLES, OVERVIEW, DEFAULTS, OPTIONS, EXIT STATUS, ENVIRONMENT, FILES, STANDARDS, SEE ALSO, HISTORY, BUGS

Digita il comando `cd ~` e descrivi cosa succede.
Perché è successo?

Quando digiti **cd ~** la shell ti porta nella **directory home dell'utente corrente**.
Succede perché cd (change directory) cambia la directory di lavoro, e il simbolo **~** è un'abbreviazione speciale della shell che rappresenta **la home dell'utente**.
La shell lo espande automaticamente prima di eseguire il comando.
Quindi cd ~ diventa di fatto qualcosa come:

cd /home/username
o, su alcuni sistemi:
cd /Users/username

In pratica torna sempre alla "base operativa" nel filesystem, indipendentemente da dove ti trovavi prima. **È una scorciatoia molto usata proprio per questo**

i. Fino a questo punto, abbiamo usato *percorsi completi o assoluti*. Percorso assoluto è il termine usato per riferirsi a percorsi che iniziano sempre dalla directory radice (/). È anche possibile lavorare con *percorsi relativi*. I percorsi relativi riducono la quantità di testo da digitare. Per capire i percorsi relativi, dobbiamo capire le directory . e .. (punto e doppio punto). Dalla directory cyops_folder3, esegui un ls –la:

```
[analyst@secOps ~]$ ls -la /home/analyst/cyops_folder3
    total 12
    drwxr-xr-x 3 analyst analyst 4096 Aug 16 15:04 .
    drwxr-xr-x 20 analyst analyst 4096 Aug 16 15:02 ..
    drwxr-xr-x 2 analyst analyst 4096 Aug 16 15:04 cyops_folder4
```

L'opzione **-a** dice a **ls** di mostrare tutti i file. Nota le voci . e .. mostrate da ls. Queste voci sono usate dal sistema operativo per tracciare la directory corrente (.) e la directory genitore (..) Puoi vedere l'uso di . e .. quando usi il comando **cd** per cambiare directory. Usare il comando cd per cambiare la directory nella directory . non comporta alcun cambiamento visibile della directory poiché . punta alla directory corrente stessa.

j. Cambia la directory corrente in /home/analyst/cyops_folder3:

```
[analyst@secOps ~]$ cd /home/analyst/cyops_folder3
[analyst@secOps cyops_folder3]$
```

k. Digita cd .

```
[analyst@secOps cyops_folder3]$ cd .
[analyst@secOps cyops_folder3]$
```

Cosa succede?

l. Cambiare la directory nella directory .., cambierà alla directory che è un livello sopra. Questa directory è anche conosciuta come *directory genitore*. Digita `cd ..`

```
[analyst@secOps cyops_folder3]$ cd ..
[analyst@secOps ~]$
```

Cosa succede?

Quale sarebbe la directory corrente se eseguissi il comando cd .. da [analyst@secOps ~]$?

Quale sarebbe la directory corrente se eseguissi il comando cd .. da [analyst@secOps home]$?

Quale sarebbe la directory corrente se eseguissi il comando cd .. da [analyst@secOps /]$?

25

Digita cd, Cosa succede?

Non succede nulla di visibile: **rimani nella stessa directory**.

Cambiare la directory nella directory .., cambierà alla directory che è un livello sopra. Questa directory è anche conosciuta come *directory genitore.* Digita `cd ..`

Cosa succede?

Ritona alla directory parente
analyst@secOps cyops_folder3]$ cd ..
[analyst@secOps ~]$ pwd
/home/analyst

Quale sarebbe la directory corrente se eseguissi il comando cd .. da [analyst@secOps~]$?

Da **[analyst@secOps ~]$** con cd .. passi alla directory **/home**, perché ~ corrisponde a **/home/analyst** e .. porta alla directory genitore.
[analyst@secOps ~]$ cd ..
analyst@secOps home]$

Quale sarebbe la directory corrente se eseguissi il comando cd .. da [analyst@secOps home]$?

Da **[analyst@secOps home]$** con cd .. arrivi alla directory **/**, che è la **root** del filesystem.
[analyst@secOps home]$ cd ..
[analyst@secOps /]$

Quale sarebbe la directory corrente se eseguissi il comando cd .. da [analyst@secOps /]$?

Da **[analyst@secOps /]$** con cd .. resti ancora in **/**, perché la root non ha una directory superiore. In Unix la root è il punto più alto dell'albero delle directory.
[analyst@secOps /]$ cd ..
[analyst@secOps /]$

Usa l'operatore > per redirigere l'output di echo a un file di testo invece che allo schermo

analyst@secOps ~]$ echo Questo è un messaggio inviato al terminale da echo. > some_text_file.txt

Nessun output è stato mostrato.

È previsto? Spiega.

```
[analyst@secOps ~]$ echo Ho vinto un Milion!  > testo.txt
[analyst@secOps ~]$ cat testo.txt
Ho vinto un Milion!
```

L'operatore **>** redirige l'output del comando verso un **file** invece che verso lo schermo. Quindi il risultato di echo non viene mostrato nel terminale, ma viene scritto dentro **testo.txt**.

f. Usa di nuovo l'operatore **>** per redirigere un diverso output di echo al file di testo **some_text_file.txt**:

```
[analyst@secOps ~]$ echo Questo è un messaggio DIVERSO, ancora una
volta inviato al terminale da echo. > some_text_file.txt
```

g. Ancora una volta, usa il comando cat per visualizzare il contenuto del file di testo **some_text_file.txt**:

```
[analyst@secOps ~]$ cat some_text_file.txt
    Questo è un messaggio DIVERSO, ancora una volta inviato al terminale da
echo.
```

Cosa è successo al file di testo? Spiega.

```
[analyst@secOps ~]$ echo Mi dai un milione di Euro  > testo.txt
[analyst@secOps ~]$ cat testo.txt
Mi dai un milione di Euro
```

Cosa è successo al file di testo? Spiega.

Il contenuto precedente del file è stato **sovrascritto**.

L'operatore **>** non aggiunge testo al file: **lo ricrea o lo sostituisce** con il nuovo output del comando. Quindi quando ho eseguito:echo Mi dai un milione di Euro > testo.txt

il contenuto precedente (**"Ho vinto un Milion!"**) è stato cancellato e rimpiazzato con la nuova frase. Per questo cat testo.txt mostra solo **"Mi dai un milione di Euro"**.

```
[analyst@secOps ~]$ echo Mi dai 2233 milione di Euro  >> testo.txt
[analyst@secOps ~]$ cat testo.txt
Mi dai un milione di Euro
Mi dai 2233 milione di Euro
```

Cosa è successo al file di testo? Spiega.

Il contenuto del file **non è stato sovrascritto**, ma **è stato aggiunto alla fine**.
L'operatore **>>** accoda l'output al file mantenendo il contenuto già presente. Quindi la nuova frase:
Mi dai 2233 milione di Euro è stata **aggiunta sotto quella già esistente**.
Per questo cat testo.txt mostra entrambe le righe.

Quanti file in più vengono visualizzati rispetto a prima? Spiega.
152 file nascosti risultano

[analyst@secOps ~]$ ls -l
total 52

[analyst@secOps ~]$ ls -la
total 204

È possibile nascondere intere directory aggiungendo un punto prima del loro nome? Ci sono directory nascoste nell'output di ls -la sopra?

Su In Linux/Unix **file e directory che iniziano con un punto (.)** sono considerati **nascosti**. Non vengono mostrati dai comandi standard come ls quando usati senza opzioni.

Alcune directory Nascoste:

1. drwx------ 4 analyst analyst 4096 Jun 19 2025 .gnupg
2. drwxr-xr-x 3 analyst analyst 4096 Mar 20 2018 .vim
3. drwxr-xr-x 2 analyst analyst 4096 Mar 24 2018 .idlerc

Identifica i parametri nel comando cp sopra.

Quali sono i file sorgente e destinazione? (usa percorsi completi per rappresentare i parametri)

Con cp il **file sorgente (SOURCE)** è il file che vuoi copiare, mentre il **file destinazione (DEST)** è il percorso dove verrà creata la copia.
Esempio con **percorsi completi**:
cp /home/analyst/file1.txt /home/analyst/cyops_folder3/file1.txt
Qui:

- **Sorgente:** /home/analyst/file1.txt
- **Destinazione:** /home/analyst/cyops_folder3/file1.txt

Quale comando hai usato per completare l'attività?

Se invece l'obiettivo è **muovere (spostare) un file**, il comando corretto non è cp ma **mv**:
mv /home/analyst/file1.txt /home/analyst/cyops_folder3/

Quali sono i vantaggi dell'utilizzo della riga di comando Linux?

La riga di comando Linux offre **velocità e controllo**. Permette di eseguire operazioni molto rapidamente senza interfaccia grafica, soprattutto quando si lavora con molti file o sistemi remoti.

Consente anche **automazione**: i comandi possono essere inseriti in script per eseguire operazioni ripetitive in modo automatico.

Infine è fondamentale per **amministrazione di sistema e cybersecurity**, perché molti server Linux funzionano senza interfaccia grafica e vengono gestiti direttamente dal terminale.